From a deterministic extraction to a probability bound

SpqrBridge slides, randomness

SpqrBridge

November 2026

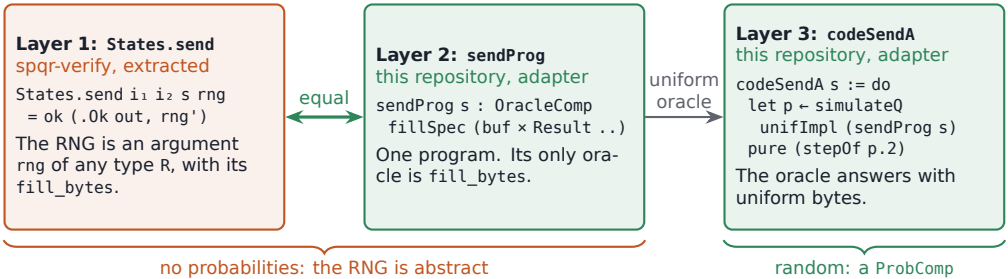
spqr-verify has no probabilities, and secure-messaging bounds a probability. How does the bridge connect them?

Two worlds, one missing fact



- The extraction is deterministic. A probability bound needs a distribution on the RNG.
- The bridge models the RNG as an *oracle*: a send calls `fill_bytes`, and the game answers each call with uniform bytes.
- Only `send` reads random bytes. `recv` is a plain function.

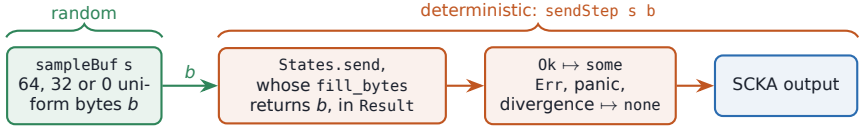
Three layers of one send



- `send_eq_prog` (proved): layer 2, run with any RNG, is layer 1.
- In layer 2, only the calls to `fill_bytes` are written by hand. The rest is the extracted code.
- The game and the theorem use layer 3.

Where the code starts running in `ProbComp`

A send of the code: `codeSendA s = do let b ← sampleBuf s; pure (sendStep s b)`
Only `sampleBuf`, the uniform answer of the oracle, is random. The rest is a function of *b*.



spqr-verify

Open, #785 `send_total`: if *b* has length `sendLen s`, a reachable send returns `Ok`.

Open, #787 `send_refines_bytes`: the matching `Braid` step uses the bytes *b*.

this repository

`codeSend_eq_some_iff`: `some` iff `Ok`.

`send_eq_prog`: the program is the extracted code.

`send_chunked`: on the same *b*, the sends give related results.

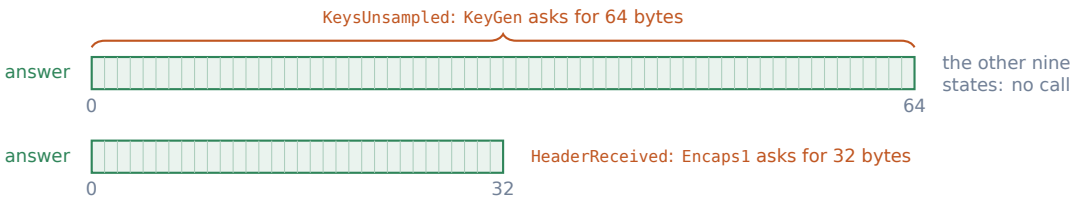
secure-messaging (SM)

`SPQR.Chunked.send` samples its own bytes. With `codeParams`: 64 or 32 bytes.

“Some randomness” is not enough

proved: `send_refines`
If the code’s send returns `Ok`, then for *some* bytes r the step `Braid.send σ r` gives a related state, the same message and the same key.

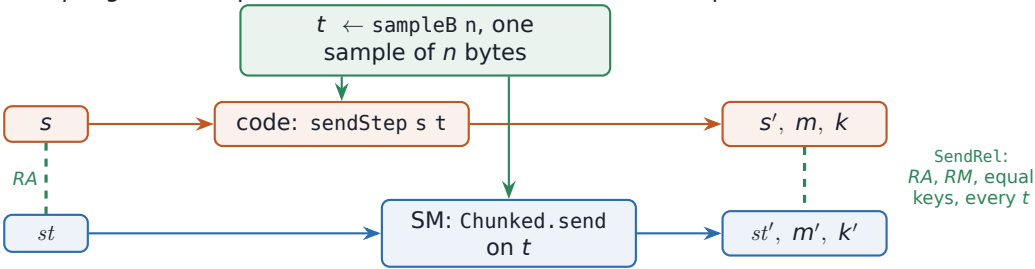
open, #787: `send_refines_bytes`
The same, with r named: r is the buffer that `fill_bytes` returned to the code.



- `send_refines` also holds for a code that ignores its RNG: take its fixed bytes as r. So it gives no distribution for r.
- With r the answer of the uniform oracle, `KeyGen` and `Encaps1` run on uniform bytes, as in SM’s model.

Coupling the two sends on one sample

A *coupling* is one experiment that runs both sends, and keeps the distribution of each.



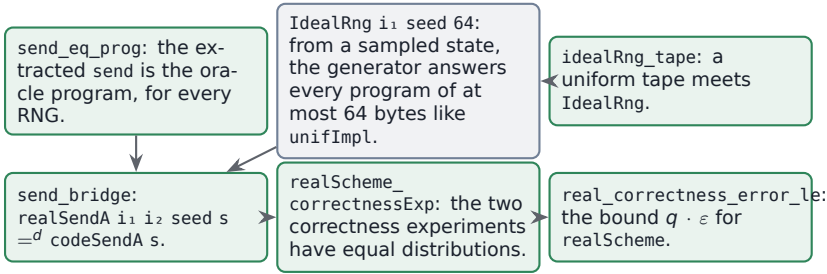
- `send_chunked`: `RelTriple (codeSendA s) (sendA st) (SendRel RA RM)`.
- Both sends use the same sample t. SM's send samples $n = 64$ or $n = 32$ bytes, as the code does, because `codeParams` builds SM's `KeyGen` and `Encaps1` from the code's primitives. With $n = 0$, neither side samples.
- The transfer theorem needs this coupling to move SM's bound to the code (main deck, "Moving the bound to the code").

Failure, the real RNG, and leaked randomness

failure in Lean	The adapter maps `Ok` to `some`, and an error, a panic or divergence to `none`. `send_total` (open, #785) rules out `none` on a reachable send, for a buffer of length `sendLen s`. It does not say that `fill_bytes` succeeds.
real RNG assumed	Given the earlier calls, `unifImpl` answers each call with uniform bytes. `IdealRng` states this assumption in Lean for a generator (next slide). No theorem connects it to `OsRng`.
security later	The security game leaks the randomness of a send (`codeSendRleak`). The coupling above ignores the leak. The security proof needs the joint distribution of leak, message, key and next state.

- The first row is a premise of the theorem. The second row is a hypothesis of `real_correctness_error_le`, and `OsRng` stays outside it.
- The third row is open work for the transfer of the security proof.

The RNG assumption as a Lean hypothesis



- `realScheme` samples a new state from `seed` for each send and drops the final state. This choice is part of the scheme. It is not a theorem.
- A tape of 64 bytes cannot answer two sends of 64 bytes. So the result does not cover one generator over a whole run.
- No theorem connects `IdealRng` to `OsRng`.